



---

# RS6xxx Firmware Download Development Guide

Number of Pages: 38  
Document Version: 1.10  
Release Date: 2026-08-31

# Revision History

| Version | Revision Description                                                                                                                                  | Date       |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------|
| 1.10    | 1. Removed the operations for obtaining and setting the clock frequency<br>2. Added instructions for multiple radars on SPI to enter programming mode | 2026-01-28 |
| 1.9     | Updated the programming procedure                                                                                                                     | 2025-08-25 |
| 1.8     | Separated the I2C programming procedure from the UART and SPI programming procedures                                                                  | 2025-05-21 |
| 1.7     | Added the Flash erase API and usage examples to DownloadLib                                                                                           | 2025-04-08 |
| 1.6     | Updated the programming flowcharts                                                                                                                    | 2025-03-18 |
| 1.5     | Merged the BurnTool User Manual into Chapter 4 of this document;<br>Merged the DownloadLib User Manual into Chapter 5 of this document;               | 2025-02-11 |
| 1.4     | Added recommendations for a failure fallback mechanism to Chapter 3;<br>Added a note to Section 3.3 stating that the last 8 KB must not be erased;    | 2024-07-20 |
| 1.3     | Added PB0 or PB2 as selectable UPD pins in Section 3.1                                                                                                | 2024-07-02 |
| 1.2     | Updated Section 3.1 by extending the level-hold time to provide additional margin for small capacitors on the line                                    | 2024-06-07 |
| 1.1     | Updated Section 3.1 by adding level-hold timing details and a programming-mode procedure for designs without NRST                                     | 2024-05-28 |
| 1.0     | Created the document template.                                                                                                                        | 2024-05-15 |

# Table of Contents

|                                                              |     |
|--------------------------------------------------------------|-----|
| Revision History .....                                       | ii  |
| Table of Contents .....                                      | iii |
| List of Figures .....                                        | v   |
| List of Tables .....                                         | vi  |
| 1 Overview .....                                             | 1   |
| 1.1 Overview .....                                           | 1   |
| 1.1.1 Development Programming .....                          | 1   |
| 1.1.2 Online Upgrade .....                                   | 1   |
| 1.1.3 Production Programming .....                           | 2   |
| 2 Programming Command Description .....                      | 3   |
| 2.1 Frame Format .....                                       | 3   |
| 2.1.1 Command Frame (CMD) .....                              | 3   |
| 2.1.1.1 Command Set .....                                    | 4   |
| 2.1.2 Response Frame (ACK) .....                             | 4   |
| 2.1.2.1 Error Codes .....                                    | 5   |
| 2.2 Command Description .....                                | 5   |
| 2.2.1 Write SRAM .....                                       | 5   |
| 2.2.2 Read SRAM .....                                        | 6   |
| 2.2.3 Erase Flash .....                                      | 7   |
| 2.2.4 Write Flash .....                                      | 7   |
| 2.2.5 Read Flash .....                                       | 8   |
| 2.2.6 Obtain Flash ID .....                                  | 9   |
| 3 Programming Procedure Description .....                    | 10  |
| 3.1 Enter Programming Mode .....                             | 10  |
| 3.1.1 Single Radar Entering Programming Mode .....           | 10  |
| 3.1.2 Multiple Radars on SPI Entering Programming Mode ..... | 10  |
| 3.2 Synchronization .....                                    | 11  |
| 3.3 Flash Erase Operation .....                              | 12  |
| 3.4 Flash Write Operation .....                              | 13  |
| 3.5 Flash Read Operation .....                               | 14  |
| 3.6 Programming Failure Retry .....                          | 15  |
| 4 BurnTool .....                                             | 17  |
| 4.1 Download Image File .....                                | 17  |
| 4.2 Developer Tools .....                                    | 18  |
| 4.2.1 SRAM Read/Write .....                                  | 18  |
| 4.2.2 Memory Read/Write .....                                | 19  |
| 4.2.3 Memory Bit Operations .....                            | 20  |
| 4.2.4 Flash Read/Write .....                                 | 20  |
| 4.2.5 Flash Erase .....                                      | 20  |
| 4.2.6 Set PC .....                                           | 21  |
| 4.2.7 Obtain System Information .....                        | 21  |
| 4.2.8 Settings .....                                         | 22  |
| 4.3 FAQs .....                                               | 23  |
| 4.3.1 Why Is the Device Serial Port Not Detected? .....      | 23  |
| 4.3.2 Why Does the Connection Keep Failing? .....            | 23  |
| 4.3.3 Why Does Erasing Keep Failing? .....                   | 23  |
| 5 DownloadLib .....                                          | 24  |
| 5.1 API Usage Instructions .....                             | 24  |
| 5.1.1 API Description .....                                  | 24  |
| 5.1.2 Return Results and Error Codes .....                   | 25  |

---

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 5.2 Usage Procedure and Examples .....                             | 26 |
| 5.2.1 Usage Procedure .....                                        | 26 |
| 5.2.2 Calling the Firmware Download API in C# .....                | 26 |
| 5.2.3 Example of Calling the Flash Erase API in C# .....           | 28 |
| 5.2.4 Example of Calling the Firmware Download API in Python ..... | 28 |
| 5.2.5 Example of Calling the Flash Erase API in Python .....       | 29 |
| 5.3 FAQs .....                                                     | 30 |
| 5.3.1 Why Is a Serial-Port Exception Returned? .....               | 30 |
| 5.3.2 Why Does Synchronization Keep Failing? .....                 | 30 |
| 5.3.3 Why Does Erasing Keep Failing? .....                         | 30 |

# List of Figures

|             |                                                                  |    |
|-------------|------------------------------------------------------------------|----|
| Figure 2-1  | Command Frame Structure (Example: Write Flash Command Frame)     | 3  |
| Figure 2-2  | Response Frame (ACK) Structure                                   | 4  |
| Figure 2-3  | Write SRAM Command Frame                                         | 5  |
| Figure 2-4  | Write SRAM Command: Fixed Header and Extended Header Sent First  | 6  |
| Figure 2-5  | Write SRAM Command: Data Sent Afterwards                         | 6  |
| Figure 2-6  | Read SRAM Command Frame                                          | 6  |
| Figure 2-7  | Read SRAM Response Frame                                         | 7  |
| Figure 2-8  | Erase Flash Command                                              | 7  |
| Figure 2-9  | Write Flash Command Frame                                        | 7  |
| Figure 2-10 | Write Flash Command: Fixed Header and Extended Header Sent First | 8  |
| Figure 2-11 | Write Flash Command: Data Sent Afterwards                        | 8  |
| Figure 2-12 | Read Flash Command Frame                                         | 9  |
| Figure 2-13 | Read Flash Response Frame                                        | 9  |
| Figure 3-1  | Multiple Radars Entering Programming Mode                        | 11 |
| Figure 3-2  | Successful Synchronization Response                              | 11 |
| Figure 3-3  | Baud-Rate Synchronization Procedure                              | 12 |
| Figure 3-4  | Flash Erase Procedure                                            | 13 |
| Figure 3-5  | Flash Write Procedure                                            | 14 |
| Figure 3-6  | Flash Read Procedure                                             | 15 |
| Figure 3-7  | SPI and UART Programming Failure Retry Procedure                 | 16 |
| Figure 3-8  | I2C Programming Failure Retry Procedure                          | 16 |
| Figure 4-1  | Image File Download                                              | 17 |
| Figure 4-2  | SRAM Read/Write                                                  | 18 |
| Figure 4-3  | Memory Read/Write                                                | 19 |
| Figure 4-4  | Memory Bit Operations                                            | 20 |
| Figure 4-5  | Flash Erase                                                      | 21 |
| Figure 4-6  | Settings                                                         | 22 |
| Figure 5-1  | Reference DLL                                                    | 26 |
| Figure 5-2  | Demo Main Procedure                                              | 27 |
| Figure 5-3  | Thread for Obtaining Download Progress                           | 27 |
| Figure 5-4  | Obtain Version Number                                            | 28 |
| Figure 5-5  | Flash Erase Example (C#)                                         | 28 |
| Figure 5-6  | Import the CLR Library File                                      | 28 |
| Figure 5-7  | Call the DLL                                                     | 29 |
| Figure 5-8  | DLL Information Output                                           | 29 |
| Figure 5-9  | Instantiate the Download Object                                  | 29 |
| Figure 5-10 | Call the Download API                                            | 29 |
| Figure 5-11 | Complete Python Calling Example                                  | 29 |
| Figure 5-12 | Flash Erase Example (Python)                                     | 30 |

# List of Tables

|            |                                                                   |    |
|------------|-------------------------------------------------------------------|----|
| Table 2-1  | Fixed Header Field Description .....                              | 3  |
| Table 2-2  | Extended Header Description .....                                 | 3  |
| Table 2-3  | CMD ID List .....                                                 | 4  |
| Table 2-4  | Response Frame (ACK) Field Description .....                      | 4  |
| Table 2-5  | Error Code Description .....                                      | 5  |
| Table 2-6  | Write SRAM Command Frame Extended Header Field Description .....  | 5  |
| Table 2-7  | Write SRAM Command Frame Extended Header Field Description .....  | 6  |
| Table 2-8  | Erase Flash Command Extended Header Field Description .....       | 7  |
| Table 2-9  | Write Flash Command Frame Extended Header Field Description ..... | 8  |
| Table 2-10 | Read Flash Command Frame Extended Header Field Description .....  | 9  |
| Table 4-1  | Device Serial Port Cannot Be Detected .....                       | 23 |
| Table 4-2  | Device Cannot Be Connected .....                                  | 23 |
| Table 4-3  | Device Cannot Be Erased .....                                     | 23 |
| Table 5-1  | API Description .....                                             | 24 |
| Table 5-2  | Return Results and Error Codes .....                              | 25 |
| Table 5-3  | Serial Port Exception .....                                       | 30 |
| Table 5-4  | Causes of Synchronization Failure .....                           | 30 |
| Table 5-5  | Flash Cannot Be Erased .....                                      | 30 |

# 1 Overview

---

This document describes the programming commands and procedures for RS6-series mmWave radar sensor chips and modules, the use of the BurnTool programming tool, and the use of the DownloadLib programming library. Software engineers, hardware engineers, and engineers responsible for production development should read the relevant sections as needed.

## 1.1 Overview

Firmware programming or upgrade for RS6-series mmWave sensor chips can be classified by application scenario into development programming, online upgrade, and production programming. Each is briefly described below.

Before the overview, the term "BURN" is defined as follows:

A boot program is stored in ROM in RS6-series mmWave radar sensor chips. It loads firmware into SPI Flash or internal SRAM through a communication interface (UART/I2C/SPI) to upgrade the firmware. This communication protocol and procedure are referred to as "BURN". Chapters 2 and 3 describe this protocol in detail.

### 1.1.1 Development Programming

During development, engineers need to program a new img to the development board for debugging. A PC programming tool is commonly used. Two programming tools are provided for the RS6 series:

1. BurnTool: Used specifically for img programming. For usage instructions, see Chapter 4.
2. xxxDemoTool: For no-development solutions, the programming function is integrated into the evaluation/development GUI tool, such as MotionDetectionDemoTool. For details, refer to the corresponding xxxx\_yyyy DEMO User Manual.

Both methods use the BURN communication protocol.

### 1.1.2 Online Upgrade

Two methods are commonly used to upgrade the radar img in a finished product:

1. Upgrade using BURN: In this solution, the RS6 acts as a slave. The Host receives the new radar img and directly controls the RS6 to perform the BURN upgrade. For the BURN protocol, see Chapters 2 and 3. Host reference code is located in the product-package directory "Software\_Kit\02\_SDK\host\_sample". For documentation, refer to the xxxx mmWave Radar Chip SDK Development Manual (source-code development solution) or the xxxx\_yyyy HOST Development Manual (no-development solution).
2. Upgrade using OTA: The new img is programmed to the SPI Flash backup area through the OTA protocol. Restart the system to switch between the old and new images. For no-development solutions, Host reference code is located in the product-package directory "Software\_Kit\02\_SDK\host\_sample". For documentation, refer to the xxxx\_yyyy HOST Development Manual. For source-code development solutions, refer to the xxxx mmWave Radar Chip SDK Development Manual.

Note: Differences between BURN and OTA:

1. The BURN communication procedure and protocol are fixed. The OTA communication procedure is fixed; its communication protocol is fixed in no-development solutions and is defined by the user in source-code development solutions;
2. BURN and OTA have different SPI Flash capacity requirements. BURN requires capacity equal to one maximum img, whereas OTA requires capacity equal to twice the maximum img size;

3. OTA supports version rollback, whereas BURN does not. BURN relies on its mandatory execution and the stability of the ROM code to ensure the upgrade.

### 1.1.3 Production Programming

The following production-programming solutions are commonly used:

1. If the solution consists of a Host controller and the company's chip or module, programming can be performed by the Host controller;
2. Reserve the required test points in the hardware design, connect the serial port through a fixture, and use the provided PC tool BurnTool for production programming;
3. Reserve the required test points in the hardware design, connect the serial port through a fixture, and use a self-developed PC production tool for production programming. Integrate the DownloadLib library provided with BurnTool into the PC production tool. See Chapter 5;
4. Program the Flash with a Flash programmer before surface mounting;

## 2 Programming Command Description

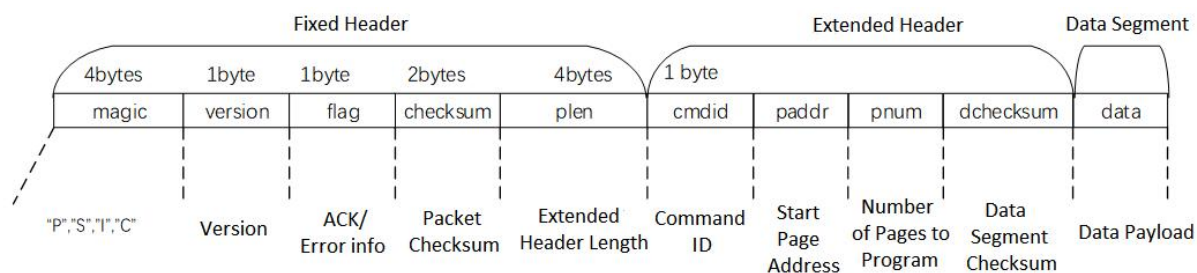
Programming requires communication with the chip to deliver the firmware. After synchronization with the chip, communication must comply with the RS6-series programming protocol. The programming protocol consists of commands and ACKs. As a basic rule, each delivered CMD corresponds to one ACK, or a CMD is split into a command header and data that are delivered separately, with each part corresponding to one ACK. The specific rule used by each command is described in this chapter.

### 2.1 Frame Format

#### 2.1.1 Command Frame (CMD)

The transmitted package consists of a fixed Header section, an Extended Header section, and a Data section. The fixed Header section consists of magic, version, flag, checksum, and plen, totaling 12 bytes. The size of the Extended Header section is determined by plen in the fixed Header section, and its structure depends on the cmdid of the specific command.

The Data field depends on dlen in the Extended Header of the specific command. It is optional and is present only in the Write SRAM and Write Flash commands.



**Figure 2-1 Command Frame Structure (Example: Write Flash Command Frame)**

**Table 2-1 Fixed Header Field Description**

| Fixed Header Field | Description                                                                                                                                                                                                                                                                                                                                                                 |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| magic              | 'PSIC', with the least-significant byte P first.                                                                                                                                                                                                                                                                                                                            |
| version            | Fixed at 1.                                                                                                                                                                                                                                                                                                                                                                 |
| flag               | Flag is 1 byte. The bits are defined as follows:<br>Bit 0: ACK flag. A value of 1 received by the PC tool indicates normal operation;<br>Bit 1: Checksum flag. Set it to 1 to require the Device to calculate the checksum;<br>Bit 2: Error flag. A value of 1 indicates an operation error;<br>Bit 3: Retry flag. A value of 1 requires the PC tool to resend the command. |
| checksum           | Bitwise complement of the u16 checksum of the fixed Header section and Extended Header section. Note: Set the checksum field to 0 during calculation.                                                                                                                                                                                                                       |
| plen               | Determines the size of the Extended Header section. The length definition depends on the command.                                                                                                                                                                                                                                                                           |

**Table 2-2 Extended Header Description**

| Extended Header Field | Description                                         |
|-----------------------|-----------------------------------------------------|
| Cmdid                 | Command ID (CMD ID). Each command uses a unique ID. |

| Extended Header Field | Description                      |
|-----------------------|----------------------------------|
| Other                 | Defined by the specific command. |

### 2.1.1.1 Command Set

Table 2-3 CMD ID List

| CMD ID(1 byte) | Description      |
|----------------|------------------|
| 0x20           | Write SRAM.      |
| 0x21           | Read SRAM.       |
| 0x30           | Erase Flash.     |
| 0x31           | Write Flash.     |
| 0x32           | Read Flash.      |
| 0x33           | Obtain Flash ID. |

### 2.1.2 Response Frame (ACK)

The Response Frame consists of a fixed Header section and a Data section.

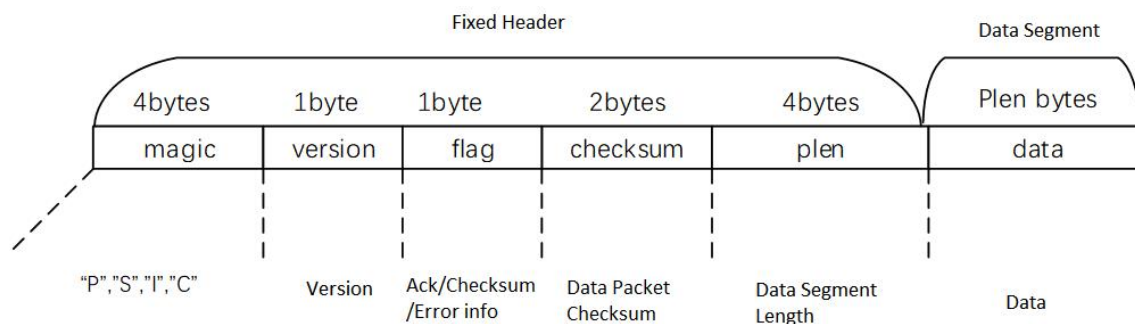


Figure 2-2 Response Frame (ACK) Structure

Table 2-4 Response Frame (ACK) Field Description

| Field    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| magic    | 'PSIC', with the least-significant byte P first.                                                                                                                                                                                                                                                                                                                                                                                                        |
| version  | Fixed at 1.                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| flag     | Flag is 1 byte. The bits are defined as follows:<br>Bit 0: ACK flag. A value of 1 received by the PC tool indicates normal operation;<br>Bit 1: Checksum flag. Set it to 1 to require the PC tool to calculate the checksum;<br>Bit 2: Error flag. A value of 1 indicates an operation error. When the Error flag is set, the Data section contains a 1-byte error code;<br>Bit 3: Retry flag. A value of 1 requires the PC tool to resend the command. |
| checksum | Bitwise complement of the u16 checksum of the fixed Header section and Data section. Note: Set the checksum field to 0 during calculation.                                                                                                                                                                                                                                                                                                              |

| Field | Description                                                               |
|-------|---------------------------------------------------------------------------|
| plen  | Determines the data length. The length definition depends on the command. |
| data  | Data area. Its content varies by command.                                 |

### 2.1.2.1 Error Codes

When the Error flag in a Response Frame is set, the Data section contains a 1-byte error code indicating that the Device failed to execute the command. The error codes are listed below:

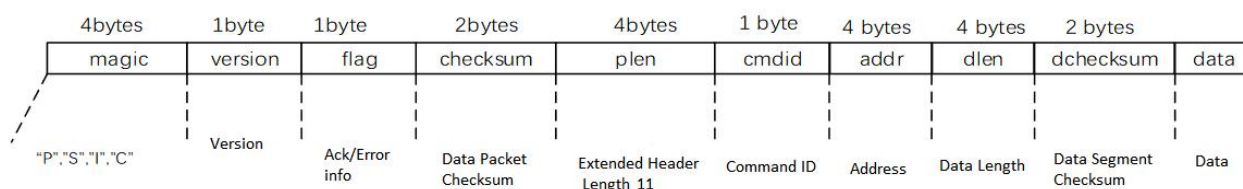
**Table 2-5 Error Code Description**

| Error Codes | Description                 |
|-------------|-----------------------------|
| 0x1         | Checksum error.             |
| 0x2         | Command transmission error. |
| 0x3         | Invalid address.            |
| 0x4         | Undefined command.          |
| 0x5         | Flash erase error.          |
| 0x6         | Flash programming error.    |
| 0x7         | Flash read error.           |
| 0x8         | Invalid data length.        |

## 2.2 Command Description

### 2.2.1 Write SRAM

The command for the Write SRAM operation is shown below. Calculate the u16 checksum of Data and enter it in Dchecksum. Calculate the u16 checksum of all fields other than Data, and enter its bitwise complement in the checksum field of the package. The command is sent in two sections: the fixed Header and Extended Header are sent as one section, and Data is sent as the other.



**Figure 2-3 Write SRAM Command Frame**

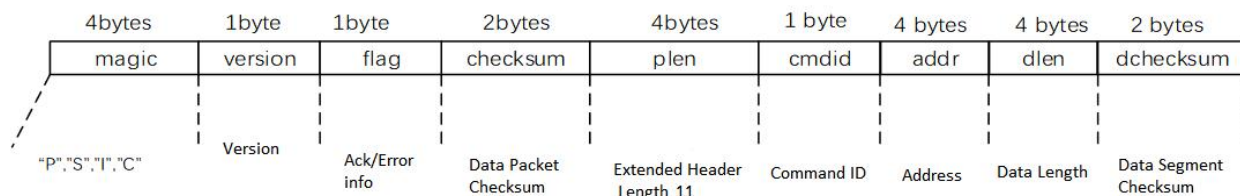
**Table 2-6 Write SRAM Command Frame Extended Header Field Description**

| Extended Header Field | Description            |
|-----------------------|------------------------|
| cmdid                 | 0x20.                  |
| addr                  | SRAM address to write. |

| Extended Header Field | Description                                                      |
|-----------------------|------------------------------------------------------------------|
| dlen                  | Data length. A Data length of no more than 16 KB is recommended. |
| dchecksum             | Checksum of the Data field.                                      |

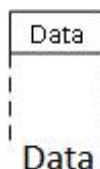
Write SRAM procedure:

- 1) Send the Header section first:



**Figure 2-4 Write SRAM Command: Fixed Header and Extended Header Sent First**

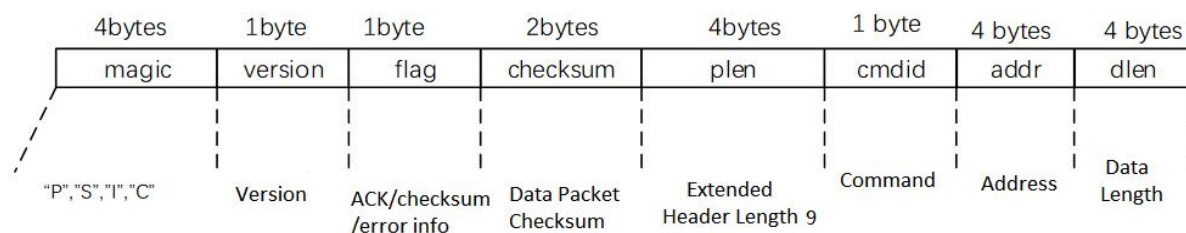
- 2) Wait for an ACK from the Device. If the command is received successfully, the Device responds as shown in Figure 2-7 Successful-Operation ACK Frame. If command reception fails, the Device responds as shown in Figure 2-5 Failed-Operation ACK Frame.
- 3) If the correct ACK is received, continue by sending the Data section:



**Figure 2-5 Write SRAM Command: Data Sent Afterwards**

- 4) Wait for an ACK, as in Step 2).

## 2.2.2 Read SRAM



**Figure 2-6 Read SRAM Command Frame**

**Table 2-7 Write SRAM Command Frame Extended Header Field Description**

| Extended Header Field | Description                                                      |
|-----------------------|------------------------------------------------------------------|
| cmdid                 | 0x21.                                                            |
| addr                  | SRAM address to read.                                            |
| dlen                  | Data length. A Data length of no more than 16 KB is recommended. |

Wait for the Device response. If the command is received successfully, the Device responds as shown below. If command reception fails, the Device responds as shown in Figure 2-5 Failed-Operation ACK Frame:

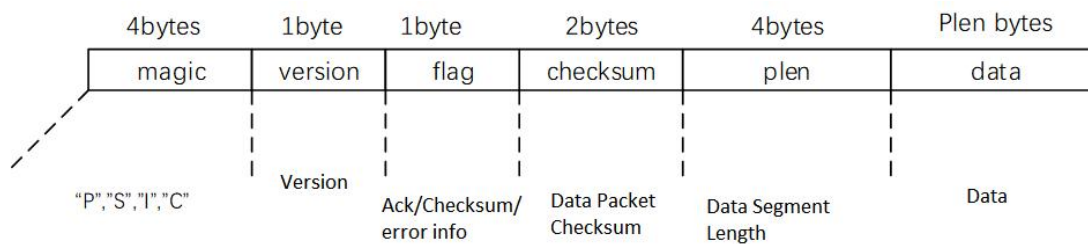


Figure 2-7 Read SRAM Response Frame

## 2.2.3 Erase Flash

The PC tool sends the erase command:

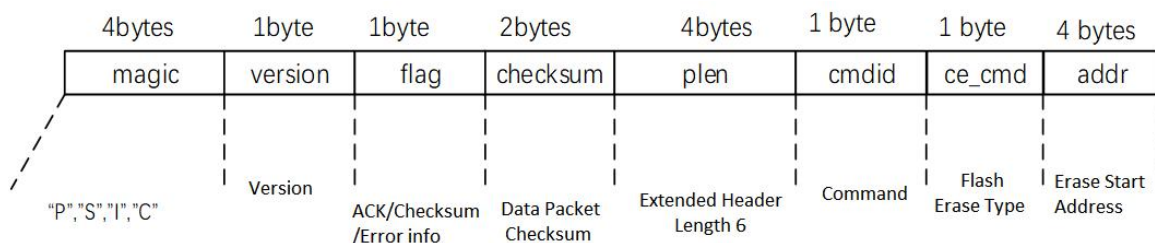


Figure 2-8 Erase Flash Command

Table 2-8 Erase Flash Command Extended Header Field Description

| Extended Header Field | Description                                                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cmdid                 | 0x30.                                                                                                                                                                                                        |
| ce_cmd                | Flash erase subcommand:<br>0x00 -- Chip erase. Note: Do not use this parameter; otherwise, chip information or user data may be lost;<br>0x01 -- 4 KB erase;<br>0x02 -- 32 KB erase;<br>0x03 -- 64 KB erase. |
| addr                  | Erase address. It must be aligned to the erase size. For example, if Ce_cmd is 0x02 for 32 KB erase, align the address to an integer multiple of 32 KB.                                                      |

After transmission, wait for an ACK from the Device. If the command is received successfully, the Device responds as shown in Figure 2-7 Successful-Operation ACK Frame. If command reception fails, the Device responds as shown in Figure 2-5 Failed-Operation ACK Frame.

## 2.2.4 Write Flash

The PC tool sends the following package for the Flash write operation:

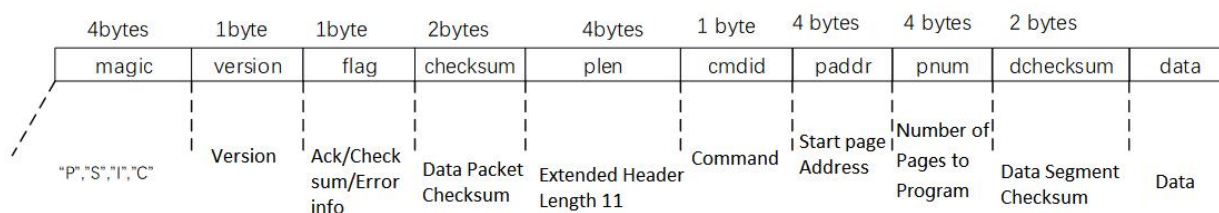


Figure 2-9 Write Flash Command Frame

Table 2-9 Write Flash Command Frame Extended Header Field Description

| Extended Header Field | Description                                                                                                                    |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| cmdid                 | 0x31.                                                                                                                          |
| paddr                 | Flash address to write. It must be aligned to an integer multiple of the Flash page size (256 bytes).                          |
| pnum                  | Number of pages. One page is 256 bytes, and the Data length is pnum * 256. A Data length of no more than 16 KB is recommended. |
| dchecksum             | u16 checksum of the Data field.                                                                                                |

Flash write procedure:

- 1) Send the command Header section first:

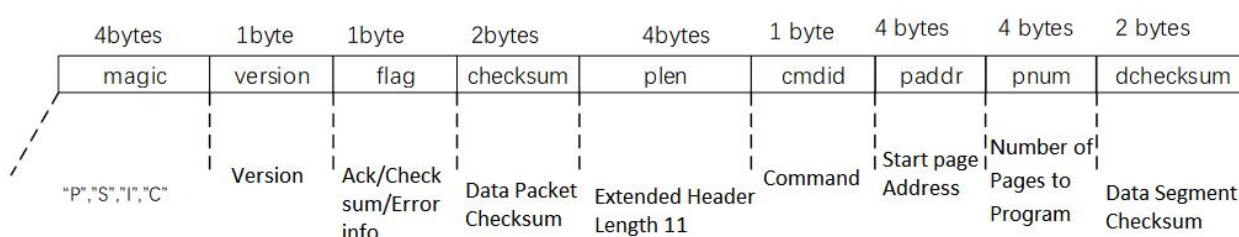


Figure 2-10 Write Flash Command: Fixed Header and Extended Header Sent First

- 2) Wait for an ACK from the Device. If the command is received successfully, the Device responds as shown in Figure 2-7 Successful-Operation ACK Frame. If command reception fails, the Device responds as shown in Figure 2-5 Failed-Operation ACK Frame.
- 3) After receiving the correct ACK, send Data:

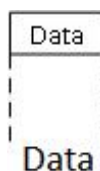


Figure 2-11 Write Flash Command: Data Sent Afterwards

- 4) Wait for an ACK, as in Step 2).

## 2.2.5 Read Flash

The PC tool sends the Flash read operation:

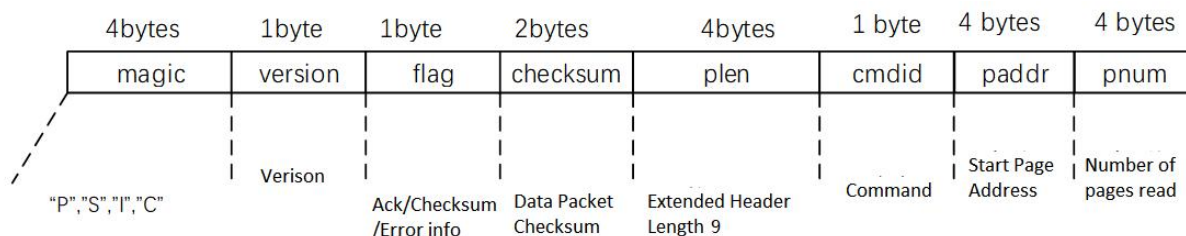


Figure 2-12 Read Flash Command Frame

Table 2-10 Read Flash Command Frame Extended Header Field Description

| Extended Header Field | Description                                                                                          |
|-----------------------|------------------------------------------------------------------------------------------------------|
| cmdid                 | 0x32.                                                                                                |
| paddr                 | Flash address to read. It must be aligned to an integer multiple of the Flash page size (256 bytes). |
| pnum                  | Number of pages to read. One page is 256 bytes. A Data length of no more than 16 KB is recommended.  |

Wait for an ACK from the Device. If the command operation succeeds, the response is as shown below. If command reception fails, the Device responds as shown in Figure 2-5

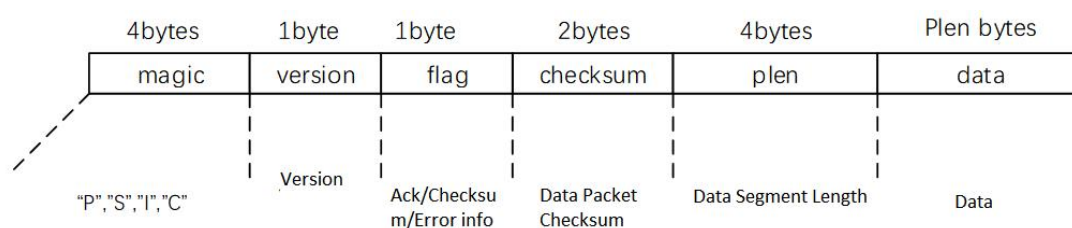


Figure 2-13 Read Flash Response Frame

### 2.2.6 Obtain Flash ID

As with obtaining the supported communication clock frequency, cmdid = 0x33. The Flash ID is also 4 bytes, with the least-significant byte first.

## 3 Programming Procedure Description

---

The basic UART and SPI programming procedure is as follows:

1. Enter programming mode;
2. Synchronize the baud rate;
3. Calculate the firmware size and erase the corresponding Flash area;
4. Write the firmware to Flash;
5. Read the firmware from Flash for verification;

The basic I2C programming procedure is as follows:

1. Enter programming mode;
2. Synchronize the baud rate;
3. Calculate the firmware size and erase the corresponding Flash area;
4. Write the firmware to Flash;

### 3.1 Enter Programming Mode

To ensure that Flash is awake when programming mode is entered, if the current radar program is running normally, send an HIF command to wake the radar and obtain the version number once before executing the procedure for entering programming mode.

#### 3.1.1 Single Radar Entering Programming Mode

To enter programming mode more reliably, either of the following two methods can be used depending on the hardware design. This method applies to UART, a single SPI slave, a single I2C slave, and multiple I2C slaves.

Method 1 applies when the Host controller can control the NRST pin through an IO. The steps are as follows:

1. Initially, PBO is high by default;
2. Pull NRST low for 20 ms, and then pull it high;
3. Wait 50 ms;
4. Pull PBO or PB2 low;
5. Pull NRST low for 20 ms, and then pull it high;
6. After waiting 50 ms, pull PBO high;

Method 2 applies when an external power-off control mechanism is available. The steps are as follows:

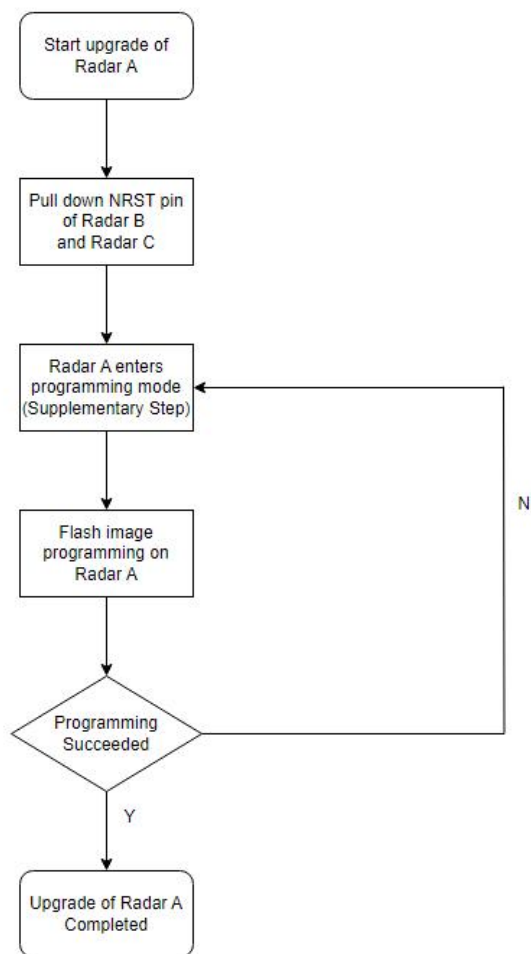
1. To prevent incomplete power-off, first wake the Device and obtain the version number;
2. Power off the company's chip and Flash, or power off the module. Note: Pull the communication pins low during power-off to prevent leakage current into the radar module.
3. Pull PBO or PB2 low;
4. For an external capacitance of 10  $\mu$ F, a 10 s power-off hold time is recommended. Adjust the power-off hold time appropriately according to the external capacitance;
5. Power on the company's chip and Flash, or power on the module again;

#### 3.1.2 Multiple Radars on SPI Entering Programming Mode

This method applies to multiple SPI slaves.

For hardware design, refer to the RS6130 Hardware Design and PCB Layout Guide.

Assume that three radars, A, B, and C, are connected to the bus. The following procedure uses upgrading radar A as an example. Upgrade B and C one at a time in the same manner.



**Figure 3-1 Multiple Radars Entering Programming Mode**

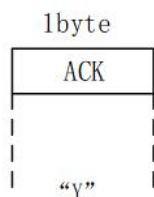
Additional information: For radar A entering programming mode, refer to Method 1 in Section 3.1.1, Single Radar Entering Programming Mode.

## 3.2 Synchronization

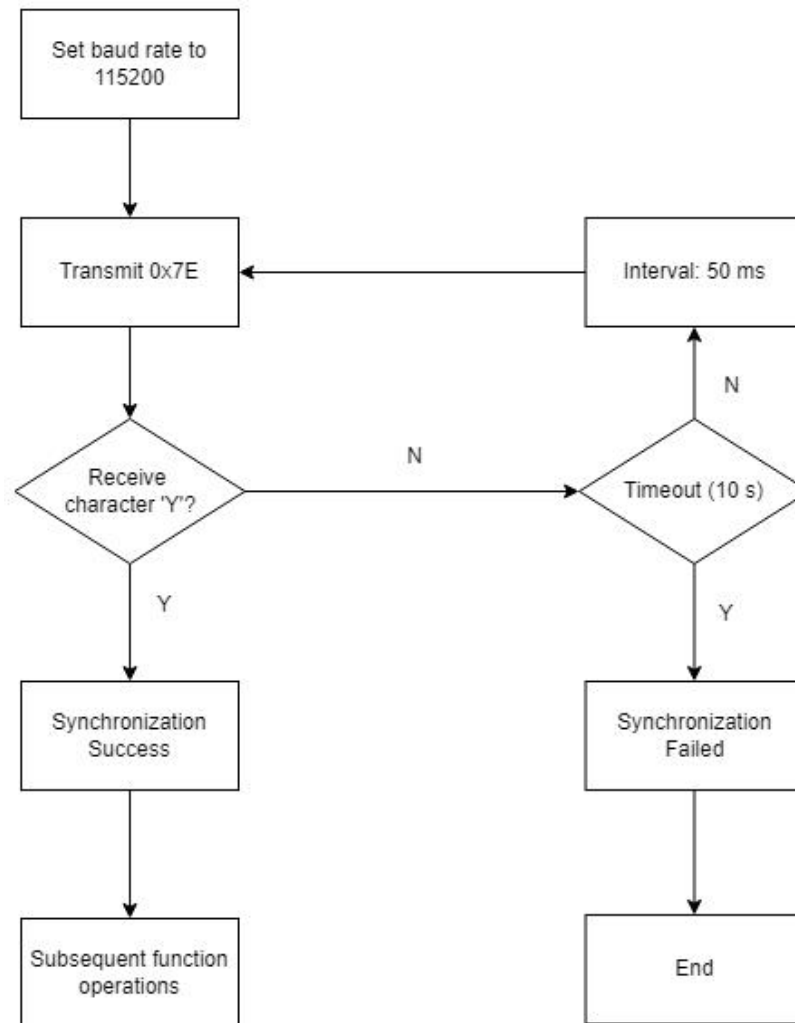
For UART communication, set the UART baud rate on the PC tool below 921600. Send 0x7E every 50 ms until 'Y' is received. Receiving 'Y' indicates successful baud-rate synchronization.

For I2C communication, the I2C address is fixed at 0x30. Set the communication rate below 400 kHz. Send 0x7E every 20 ms. If an ACK is received, read the data. If a NACK is received, send 0x7E again after 20 ms until 'Y' is received.

For SPI communication, SPI uses single-wire half-duplex mode, with Clock Polarity set to 0 and Clock Phase set to 0. Set the communication rate below 8 Mbps.



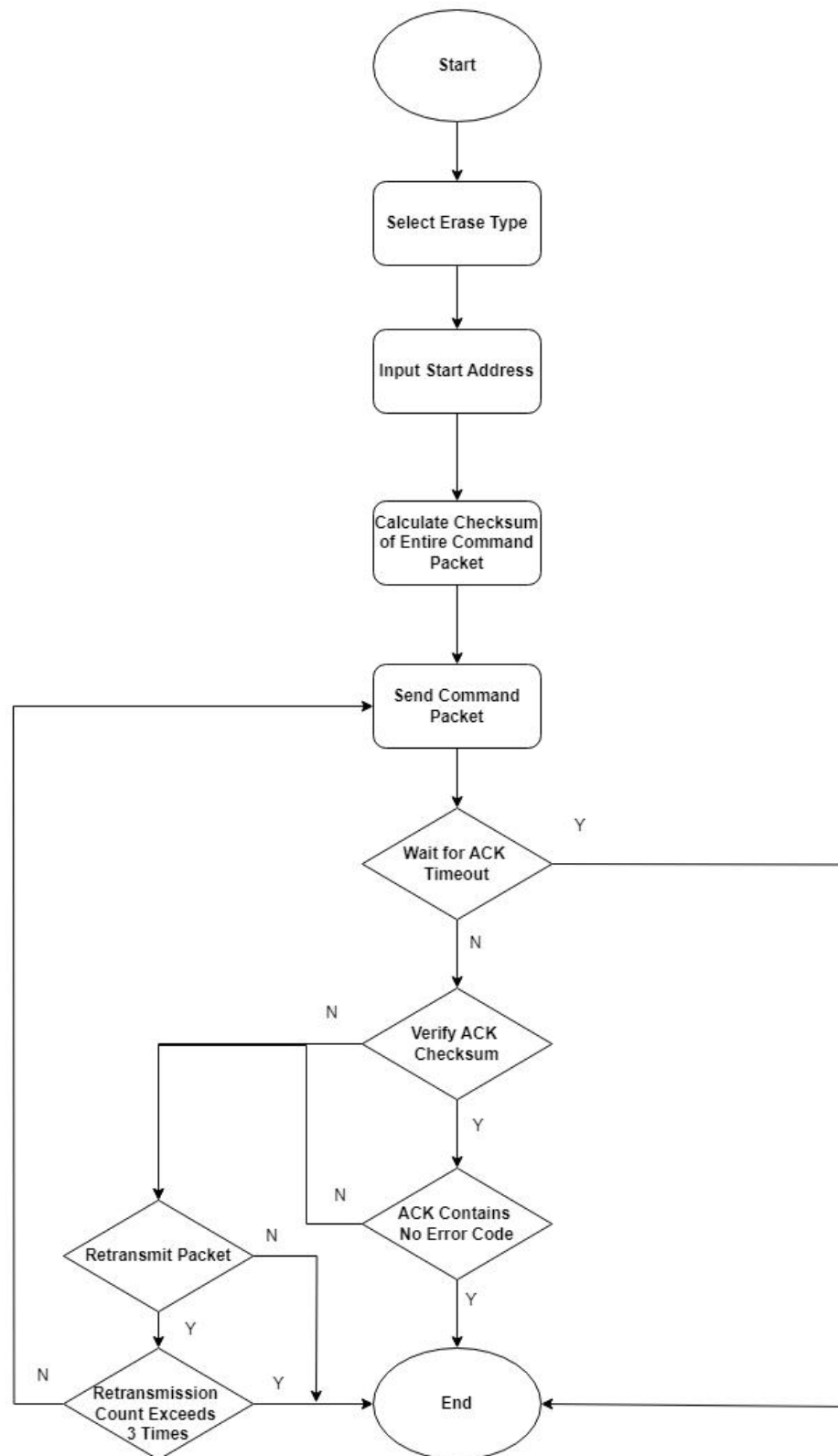
**Figure 3-2 Successful Synchronization Response**



**Figure 3-3 Baud-Rate Synchronization Procedure**

### 3.3 Flash Erase Operation

Erase the Flash area from address 0 to the firmware size rounded up to 4 KB. Use the erase command to erase each block:



**Figure 3-4 Flash Erase Procedure**

Note: The last 8 KB of Flash is generally used to store data. Do not erase it.

### 3.4 Flash Write Operation

Write data from address 0 through the entire firmware size. A write size of 4 KB per transaction is recommended. Select the write size according to the actual bit error rate.

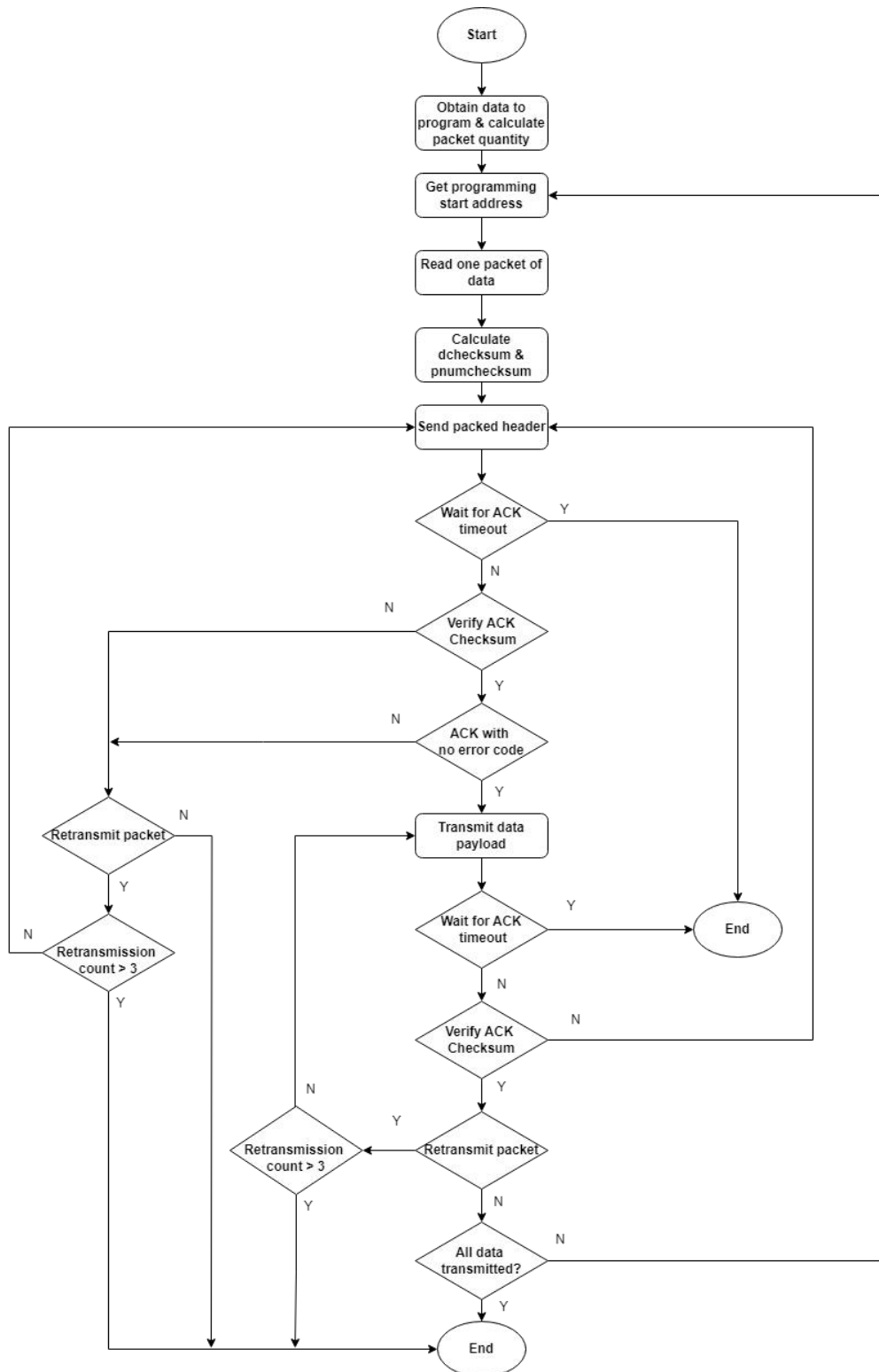


Figure 3-5 Flash Write Procedure

### 3.5 Flash Read Operation

Read data from Flash address 0 through the firmware size and compare it with the firmware to ensure that the written content is identical. Read no more than two pages per transaction.

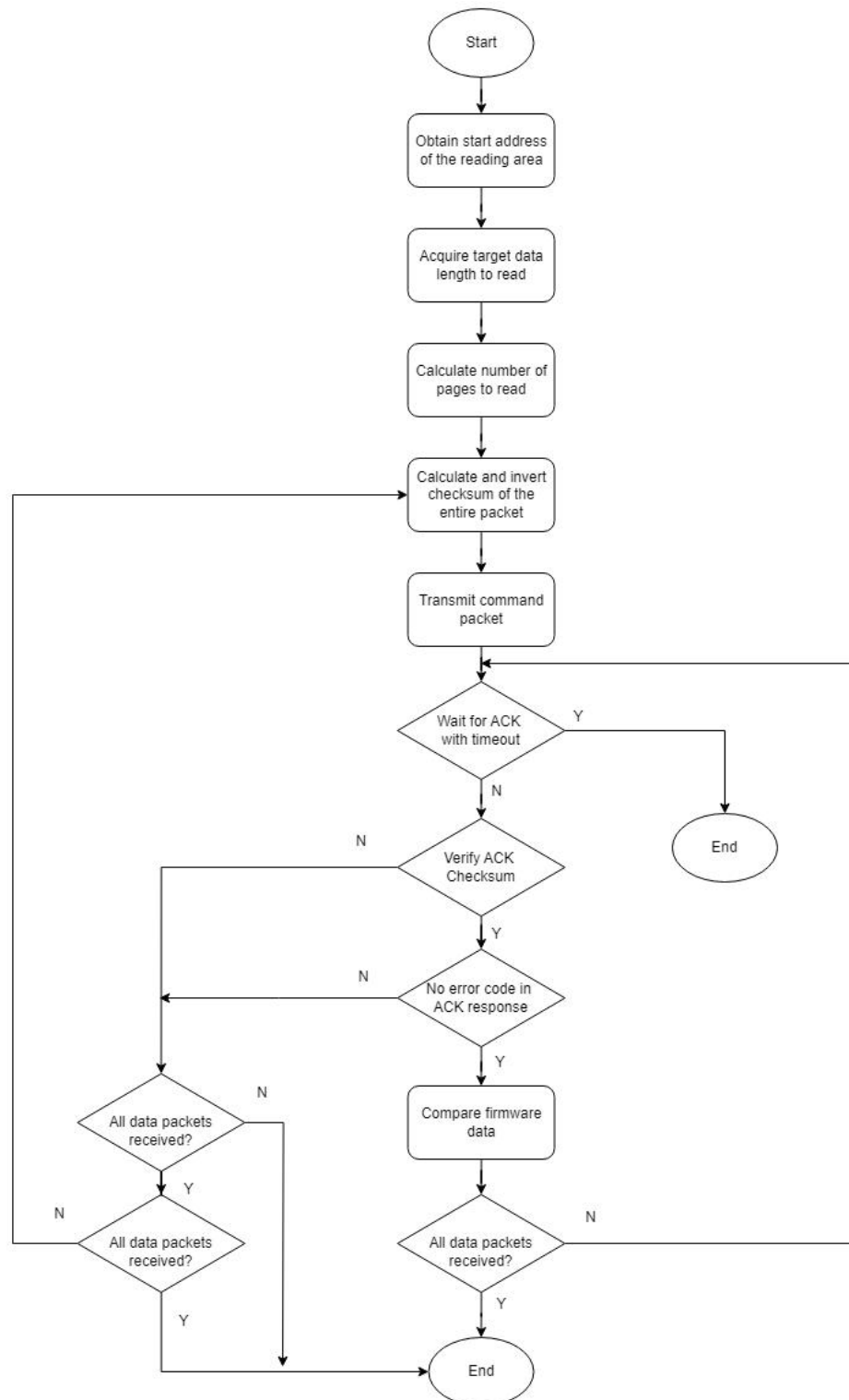
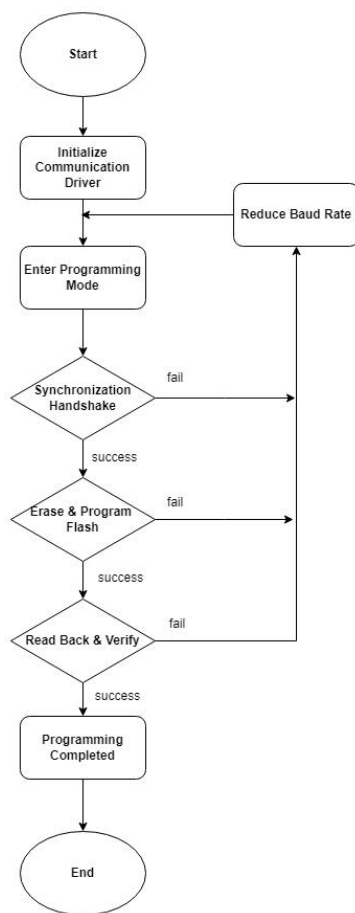


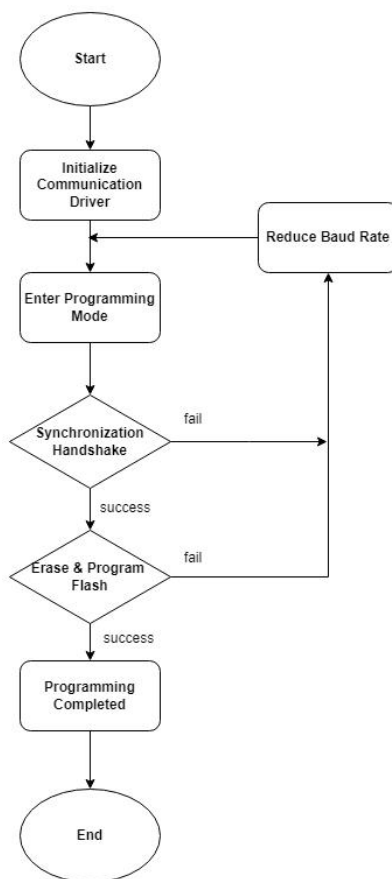
Figure 3-6 Flash Read Procedure

## 3.6 Programming Failure Retry

If an unrecoverable failure occurs during programming, repeat the complete programming procedure to ensure chip operation. First, make the chip or module enter programming mode again. Then retry at a lower baud rate to improve communication reliability.



**Figure 3-7 SPI and UART Programming Failure Retry Procedure**



**Figure 3-8 I2C Programming Failure Retry Procedure**

## 4 BurnTool

This chapter describes how to use BurnTool, the programming tool for RS6-series mmWave radar sensor chips, and provides answers to FAQs encountered during use.

This tool supports serial communication only.

BurnTool is available in the SDK software package.

- Path: [Software Kit\03 Tool\BurnTool](#)

### 4.1 Download Image File

The main interface of the programming tool is shown below:

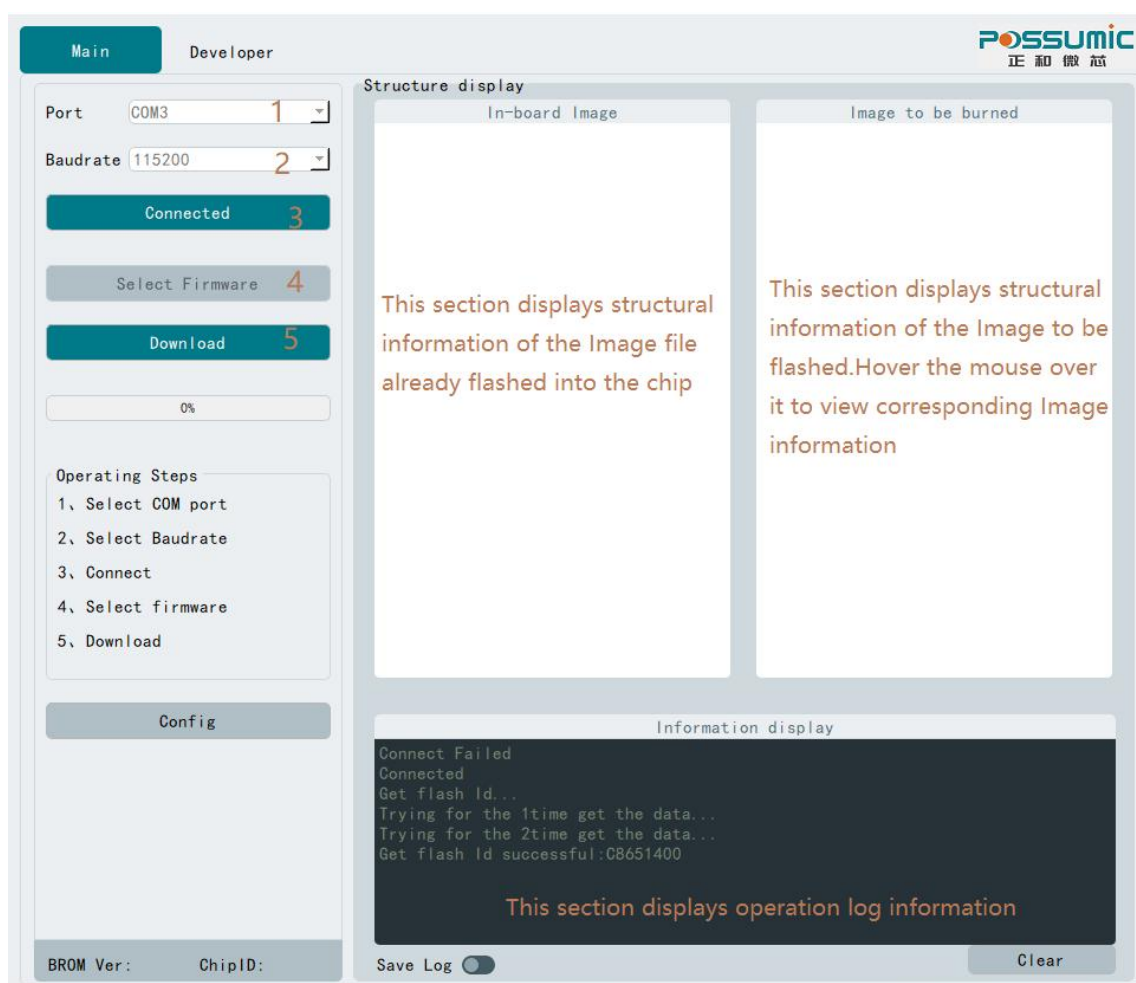


Figure 4-1 Image File Download

First, press the "NRST" button on the development board to reset the module. If an Image file has already been programmed to Flash, press the "UPD" and "NRST" buttons simultaneously. Release the buttons, and then perform the following steps:

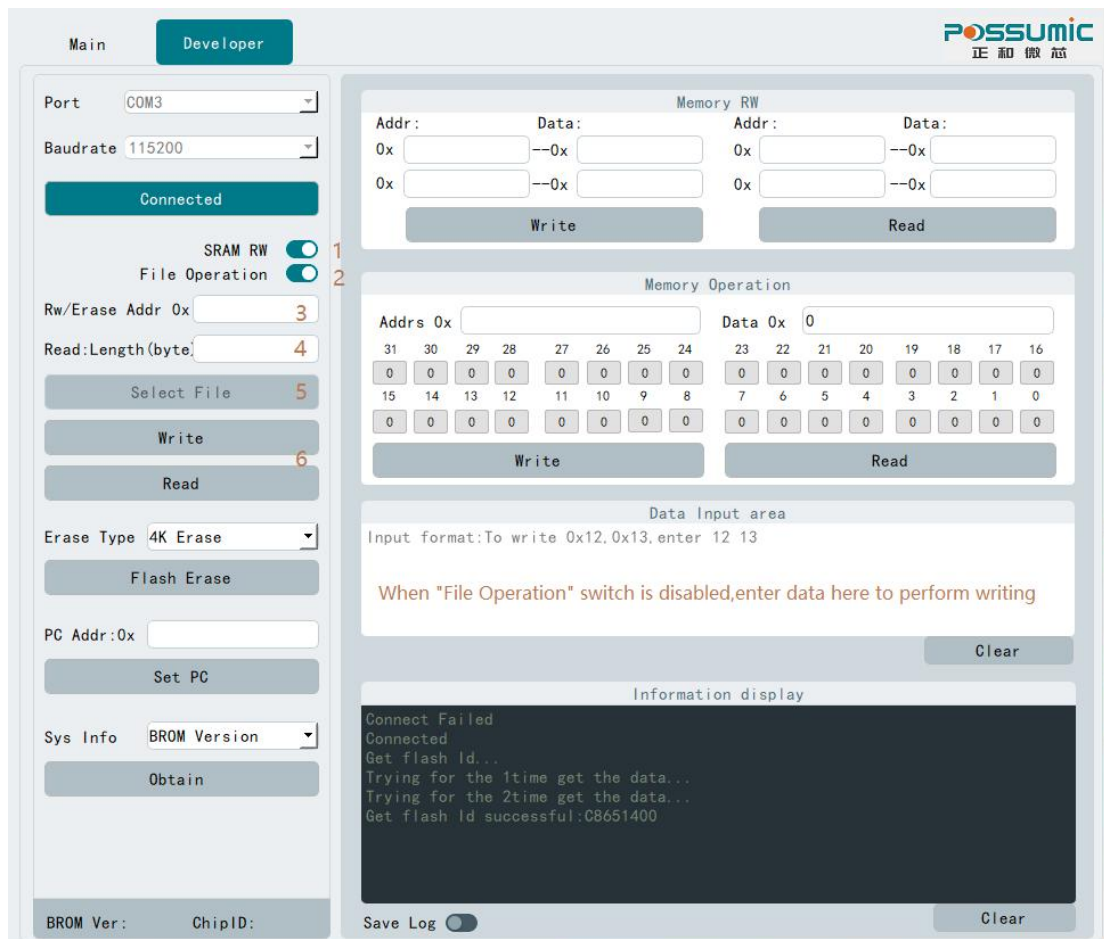
- 1、 Select the serial port of the Device to which the Image file will be programmed;
- 2、 Select the serial communication baud rate. Note: The maximum baud rate currently supported by the module is 921600;
- 3、 Click "Connect":
  - (1) Connection successful: The button turns green and displays "Connected";

- (2) Connecting: The button color remains unchanged and the button displays "Connecting...(Click to Cancel Connection)";
- (3) Connection failed: The button turns red and displays "Connection Failed";
- 4、 Click "Select Firmware" and select the Image file to download in the file-selection dialog box;
- 5、 Click "Download":
  - (1) Downloading: The progress bar and information window display the current download progress;
  - (2) Download successful: The progress bar displays 100%, and the information window displays "Download Successful" and the elapsed time;
  - (3) Download failed: The progress bar is cleared, and the information window displays "Download Failed" and the cause of the failure.

## 4.2 Developer Tools

### 4.2.1 SRAM Read/Write

As shown below:



**Figure 4-2 SRAM Read/Write**

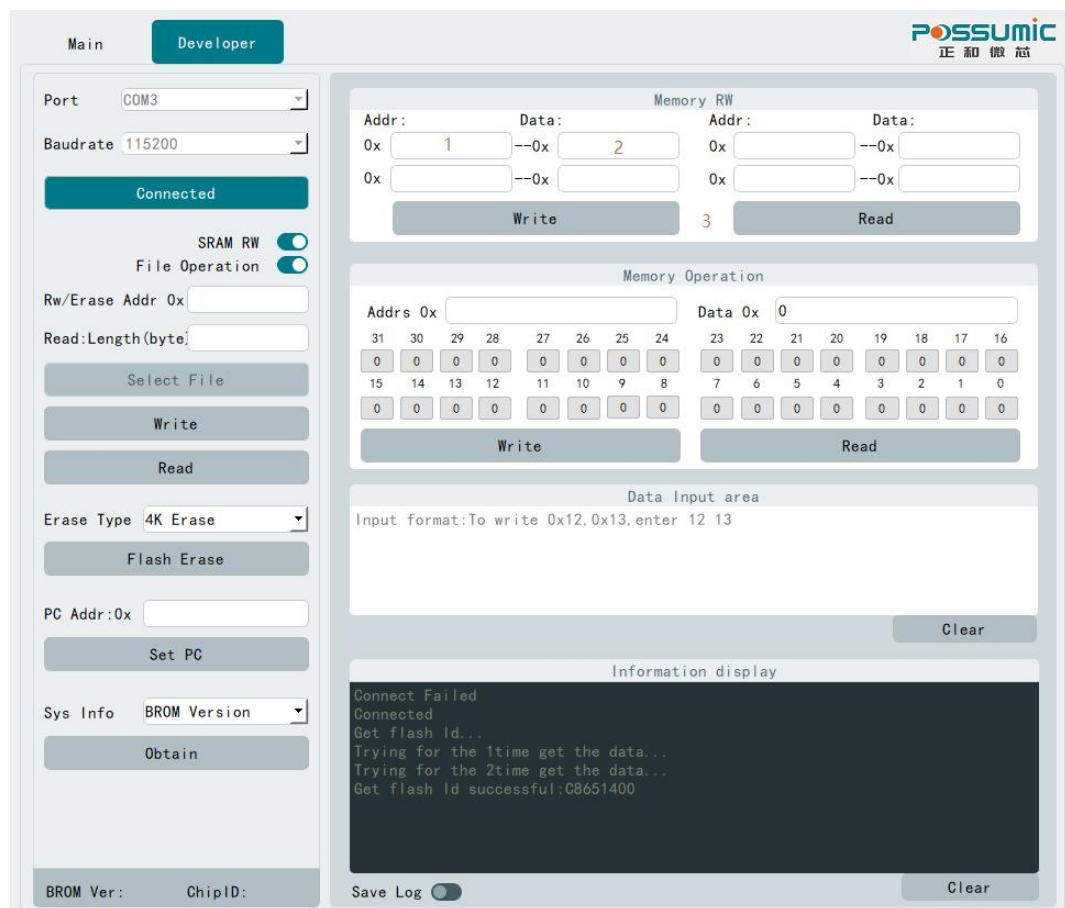
First, click "Developer Interface" in the upper-left corner to switch interfaces. If the module is not connected, follow Section 5.1.1, completing its first 3 steps to connect it, and then perform the following steps:

- 1、 Enable "SRAM Read/Write";
- 2、 To write a file, enable "File Operation";

- 3、 Enter the address to read or write;
- 4、 For a read operation, enter the length (decimal) at position 4 in the preceding figure. Unit: byte. For a write operation, skip this step;
- 5、 Select the file to write. If "File Operation" was disabled in Step 2, enter the data to write in "Data Input Area". For a read operation, skip this step;
- 6、 Click "Write" or "Read". For a read operation, if the data must be saved to a file, enable "File Operation" in Step 2 and select a save path in the file dialog box that opens when "Read" is clicked. The information window displays the operation status:
  - (1) Reading/Writing: Displays the progress;
  - (2) Read/Write successful: Displays "Write Successful" or "Read Successful";
  - (3) Read/Write failed: Displays "Write Failed" or "Read Failed" and the cause of the failure.

## 4.2.2 Memory Read/Write

As shown below:



**Figure 4-3 Memory Read/Write**

First, click "Developer Interface" in the upper-left corner to switch interfaces. If the module is not connected, follow Section 3 steps to connect it, and then per

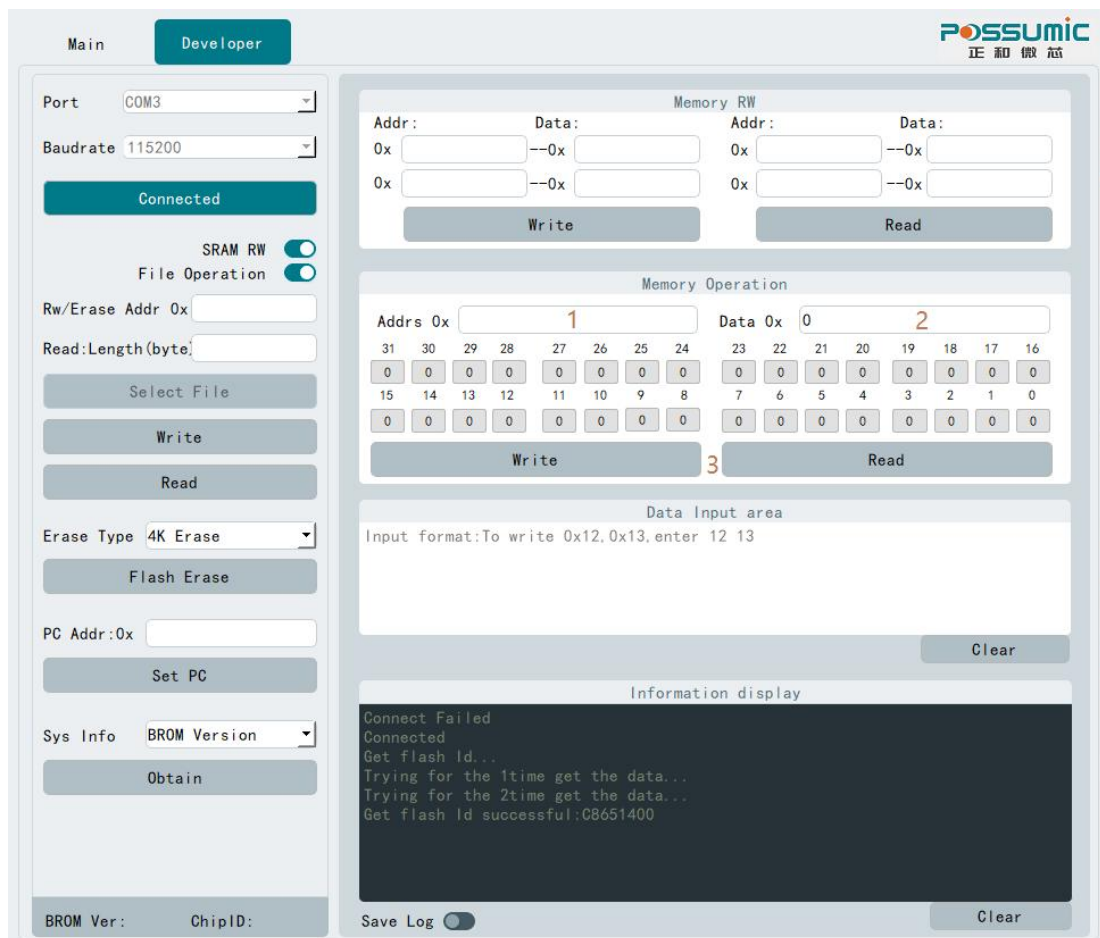
form the following steps:

- 1、 Enter the read/write address;
- 2、 Enter the data to write. For a read operation, skip this step;
- 3、 Click "Write" or "Read";

- 4、 The read/write result is displayed in the information window.

### 4.2.3 Memory Bit Operations

As shown below:



**Figure 4-4 Memory Bit Operations**

First, click "Developer Interface" in the upper-left corner to switch interfaces. If the module is not connected, follow Section 3 steps to connect it, and then perform the following steps:

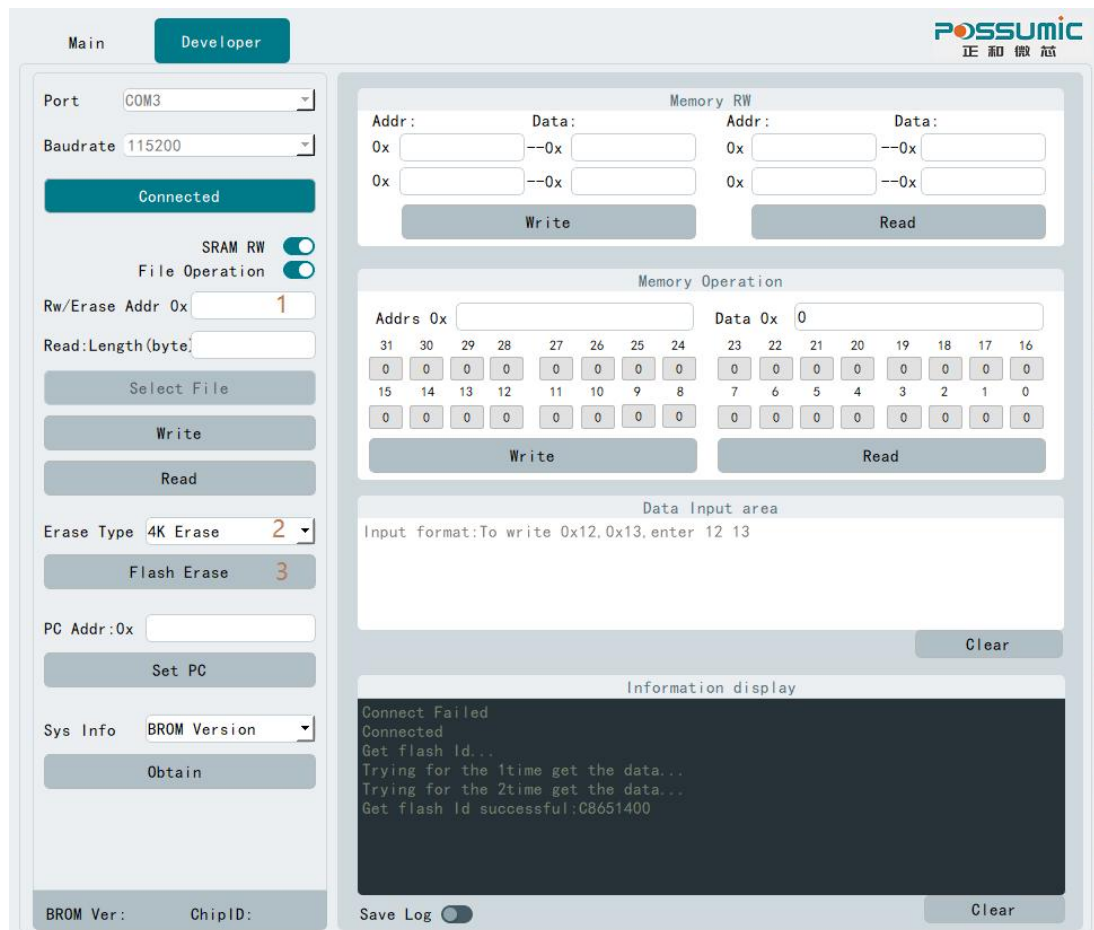
- 1、 Enter the memory address;
- 2、 Enter the data to write. You can also click each bit to enter data. For a read operation, skip this step;
- 3、 Click "Write" or "Read";
- 4、 The read/write result is displayed in the information window.

### 4.2.4 Flash Read/Write

The Flash read/write procedure follows the SRAM read/write procedure. In Step 1, disable "SRAM Read/Write".

### 4.2.5 Flash Erase

As shown below:



**Figure 4-5 Flash Erase**

First, click "Developer Interface" in the upper-left corner to switch interfaces. If the module is not connected, follow Section 3 steps to connect it, and then perform the following steps:

- 1、 Enter the start address of the erase area;
- 2、 Select the erase type;
- 3、 Click "Flash Erase";
- 4、 The information window displays the erase progress and result.

#### 4.2.6 Set PC

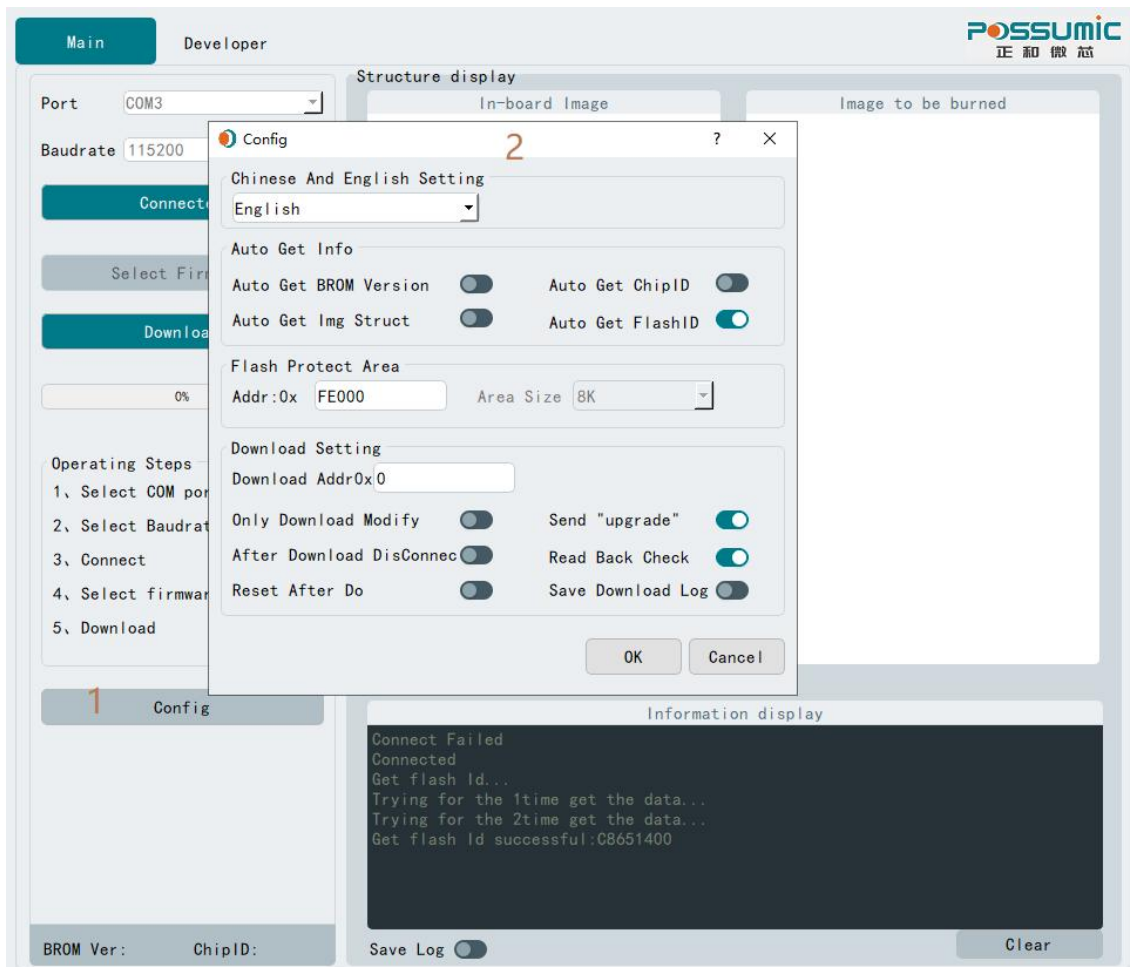
- 1、 Enter the PC address;
- 2、 Click "Set PC";
- 3、 The setting result is displayed in the information window.

#### 4.2.7 Obtain System Information

- 1、 Select the information to obtain;
- 2、 Click "Obtain";
- 3、 The obtained information is displayed in the information window.

## 4.2.8 Settings

As shown below:



**Figure 4-6 Settings**

- 1、 Click "Settings" on the main interface to open the Settings window;
- 2、 Configure the settings as required:
  - (1) Language: Switch the software interface between Chinese and English;
  - (2) Automatically Obtain Information: Automatically obtains the corresponding information after the Device is connected successfully;
  - (3) Download Settings:
    - a. Download Address: Changes the download address. Note: Only address 0 is currently supported;
    - b. Download Modified Sections Only: Downloads only the sections of the Image file that differ from the currently programmed Image file;
    - c. Send Upgrade Command: Sends the "upgrade" command before downloading to enter firmware-update mode;
    - d. Disconnect Serial Port After Download: Automatically disconnects the serial port after downloading;
    - e. Save Download Log: Saves Image download information.

## 4.3 FAQs

### 4.3.1 Why Is the Device Serial Port Not Detected?

**Table 4-1 Device Serial Port Cannot Be Detected**

| Possible Cause                                                                                 | Corrective Action                                                    |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| The serial-port driver for the Device is not installed in the current Windows operating system | Install the corresponding serial-port driver as prompted by Windows. |

### 4.3.2 Why Does the Connection Keep Failing?

Possible causes are as follows:

**Table 4-2 Device Cannot Be Connected**

| Possible Cause                                  | Corrective Action                                                                                       |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| The correct Device serial port is not selected. | Confirm that the correct Device serial port is selected, reset the module, and reconnect.               |
| The correct baud rate is not selected           | The maximum currently supported baud rate is 921600. Confirm that an appropriate baud rate is selected. |

### 4.3.3 Why Does Erasing Keep Failing?

Possible causes are as follows:

**Table 4-3 Device Cannot Be Erased**

| Possible Cause                                                                                                                                    | Corrective Action                                                                   |
|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| The previous program entered sleep mode, and programming mode was not entered correctly after the UPD button was pressed and the Device was reset | Hold down the UPD button, toggle the power switch, and enter programming mode again |

## 5 DownloadLib

DownloadLib.dll is primarily used for firmware programming of RS6-series mmWave radar sensor chips. This DLL can be called when the programming function must be integrated into a production-test tool, test system, or other PC tool.

This DLL supports UART communication only.

DownloadLib.dll is available in the SDK software package.

- Path: [Software Kit\03\\_Tool\DownloadLib\DownloadLib.dll](#)

### 5.1 API Usage Instructions

#### 5.1.1 API Description

**Table 5-1 API Description**

| Interface                                                                                                                      | Parameter                                                                                                                                                                                                                                                                                                                | Description                                |
|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| Int StartDownloadFw(string com, string baudRateStr, string fwFile, bool readBackCheck = true, bool downloadFinishStart = true) | 1. string com: Serial port number of the development board.<br>2. string baudRateStr: Serial-port baud rate.<br>3. string fwFile: Firmware path.<br>4. bool readBackCheck: Whether to perform readback verification (default: true).<br>5. bool downloadFinishStart: Whether to start after downloading (default: true). | Starts firmware download in one operation. |
| Int GetDownloadProgressValue()                                                                                                 | None                                                                                                                                                                                                                                                                                                                     | Obtains download progress.                 |
| Int GetVersion()                                                                                                               | None                                                                                                                                                                                                                                                                                                                     | Obtains the DownloadLib version.           |
| Void SetWaitAckTimeout(UInt32 timeout)                                                                                         | 1. UInt32 timeout: Maximum time to wait for an ACK (unit: ms); default: 3000 ms.                                                                                                                                                                                                                                         | Sets the ACK wait timeout.                 |
| Void SetTryAgainTimes(int retryNum)                                                                                            | 1. int retryNum: Number of retries; default: 3.                                                                                                                                                                                                                                                                          | Sets the number of retries.                |
| Int ConnectDevice(string com, string baudRateStr)                                                                              | 1. string com: Serial port number of the development board.<br>2. string baudRateStr: Serial-port baud rate.                                                                                                                                                                                                             | Configures the connection for Flash erase. |
| Void DisconnectDevice()                                                                                                        | None                                                                                                                                                                                                                                                                                                                     | Disconnects the Device.                    |

| Interface                           | Parameter                                                                                                                                                                                                                                                                | Description   |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|
| Int FlashErase(uint addr, uint len) | <p>1、addr: Start address of the Flash area to erase. Note: The address must be aligned to 4 KB; otherwise, an unknown error may occur.</p> <p>2、len: Length to erase. Note: If this value is not aligned to 4 KB, the erase length is rounded up to a 4 KB boundary.</p> | Erases Flash. |

### 5.1.2 Return Results and Error Codes

Table 5-2 Return Results and Error Codes

| Error Codes | Description                                  |
|-------------|----------------------------------------------|
| 0x59        | Download successful                          |
| 0x0         | Unknown error                                |
| 0x1         | Checksum error                               |
| 0x2         | Command error                                |
| 0x3         | Invalid address                              |
| 0x4         | Undefined command                            |
| 0x5         | Flash erase error                            |
| 0x6         | Flash programming error                      |
| 0x7         | Flash read error                             |
| 0x8         | Invalid data length                          |
| 0x9         | Firmware open error                          |
| 0xA         | Serial-port open error                       |
| 0xB         | Baud-rate synchronization connection timeout |
| 0xC         | ACK wait timeout                             |
| 0xD         | Firmware-size alignment failure              |
| 0xE         | Readback verification mismatch               |
| 0xF         | Serial-port exception                        |
| 0x10        | Erase address is not aligned to 4 KB         |
| 0x1C        | Flash erase ACK wait timeout                 |
| 0x2C        | Download ACK wait timeout                    |
| 0x3C        | Readback-verification ACK wait timeout       |

| Error Codes | Description                       |
|-------------|-----------------------------------|
| 0x4C        | Firmware-startup ACK wait timeout |

## 5.2 Usage Procedure and Examples

### 5.2.1 Usage Procedure

The usage procedure is as follows:

- 1、 Add a reference to DownloadLib.dll;
- 2、 Set the ACK wait timeout; default: 3000 ms. Skip this step if the default value is used;
- 3、 Set the number of retries; default: 3. Skip this step if the default value is used;
- 4、 Use a separate thread to continuously obtain the download progress. Skip this step if the progress is not required;
- 5、 Start the download.

### 5.2.2 Calling the Firmware Download API in C#

```
using DownloadLib;
```

Figure 5-1 Reference DLL

```

DownloadLib.Download download;
0 个引用
private void Test()
{
    string com = "COM1";
    string baudrate = "115200";
    string fwFilePath = "C:\\Users\\Desktop\\test.img";
    download = new Download();
    // Set the number of retry attempts.
    // This step can be skipped if the default value of 3 is used.
    download.SetTryAgainTimes(5);
    // Set the Ack wait timeout in milliseconds(ms).
    // The step can be skipped if the default value of 3000 is used.
    download.SetWaitAckTimeout(5000);
    // Before starting the download,
    // use a separate thread to continuously obtain the download progress.
    // This step can be skipped if download progress monitoring is not required.
    Thread getProgressThread = new Thread(RefsProgressHandler);
    getProgressThread.IsBackground = true;
    getProgressThread.Start();

    // Start download
    int result = download.StartDownloadFw(com, baudrate, fwFilePath);
    MessageBox.Show($"Download result:{result}");
}

```

Figure 5-2 Demo Main Procedure

```

private void RefsProgressHandler()
{
    int progressValue = 0;
    while (true) {
        // Get download progress
        progressValue = download.GetDownloadProgressValue();
        Console.WriteLine($"Download progress:{progressValue}%");
        if (progressValue >= 100) {
            break;
        }
        Thread.Sleep(10);
    }
}

```

Figure 5-3 Thread for Obtaining Download Progress

```

0 references | 0 changes | 0 authors, 0 modifications
private void GetVersion()
{
    int version = download.GetVersion();
    Console.WriteLine("DownloadLib Version:" + version);
}

```

Figure 5-4 Obtain Version Number

### 5.2.3 Example of Calling the Flash Erase API in C#

Calling procedure:

- 1、 Call ConnectDevice to connect the development board;
- 2、 Call FlashErase to erase Flash;
- 3、 Call DisConnect to disconnect the development board.

```
private void startEraseBtn_Click(object sender, EventArgs e)
{
    DownloadLib.Download download = new Download();
    int result = download.ConnectDevice("COM1", "115200");
    if (result != 0x59) {
        MessageBox.Show($"Connect failed, error code: {result}");
        return;
    }
    // Erase 64KB starting from Flash address 0;
    // the address must be 4KB-aligned.
    result = download.FlashErase(0, 65536);
    if (result != 0x59) {
        MessageBox.Show($"Erase failed, error code: {result}");
    } else {
        MessageBox.Show("Erase succeeded!");
    }

    download.DisConnectDevice();
}
```

Figure 5-5 Flash Erase Example (C#)

### 5.2.4 Example of Calling the Firmware Download API in Python

- 1、 Install the Python third-party libraries required to call the DLL: cffi, clr-loader, and pythonnet;
- 2、 Call the DLL:
  - (1) Place the DLL file in the current working directory;
  - (2) Import the clr-loader library into the Python script;

```
import clr
```

Figure 5-6 Import the CLR Library File

- (3) Call the DLL;

```
clr.FindAssembly("DownloadLib.dll")
dll = clr.AddReference("DownloadLib")
from DownloadLib import Download
print(dll)
```

**Figure 5-7 Call the DLL**

(4) Check the script output. If the following information is displayed, the DLL has been imported successfully.

```
DownloadLib, Version=1.0.0.0, Culture=neutral, PublicKeyToken=null
```

**Figure 5-8 DLL Information Output**

3、 Use the DLL to download firmware

(1) Instantiate the programming DLL;

```
download_instance = Download()
```

**Figure 5-9 Instantiate the Download Object**

(2) Call the programming API.

```
serial_port = "COM1"
baudrate = "115200"
firmware_path = "Test.img"
result = download_instance.StartDownloadFw(serial_port, baudrate, firmware_path)
```

**Figure 5-10 Call the Download API**

4、 Complete Example

```
import clr
clr.FindAssembly("DownloadLib.dll")
dll = clr.AddReference("DownloadLib.dll")
from DownloadLib import Download
print(dll)
download_instance = Download()
serial_port = "COM1"
baudrate = "115200"
firmware_path = "Test.img"
result = download_instance.StartDownloadFw(serial_port, baudrate, firmware_path)
if hex(result) == "0x59":
    print(firmware_path + ": Programming succeeded.")
```

**Figure 5-11 Complete Python Calling Example**

### 5.2.5 Example of Calling the Flash Erase API in Python

Calling procedure:

- 1、 Call the dynamic-link library by completing Steps 1 to 3 in Section 5.2.4;
- 2、 Flash erase API calling example:

```
def start_erase(serial_port, baudrate, start_addr, len):
    """start to erase firmware
    param:
        serial_port:[string] the serial com,sub as "COM1"
        baudrate:[int] download baudrate,sub as 115200
        start_addr:[int] start erase address
        len:[int] total erase length
    return:
        status

    """
    # instantiate DownloadLib.dll
    download_instance = Download()
    # Connect device
    result = download_instance.ConnectDevice(serial_port, baudrate)
    # Execute the following functions in programming mode
    erase_result = download_instance.FlashErase(start_addr, len)
    # Disconnect device
    download_instance.DisConnectDevice()

if __name__ == '__main__':
    print(dll)
    start_erase(serial_port="COM1", baudrate="921600", start_addr=0, len=65536)
```

Figure 5-12 Flash Erase Example (Python)

## 5.3 FAQs

### 5.3.1 Why Is a Serial-Port Exception Returned?

Table 5-3 Serial Port Exception

| Possible Cause                                                                                 | Corrective Action                                                    |
|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| The serial-port driver for the Device is not installed in the current Windows operating system | Install the corresponding serial-port driver as prompted by Windows. |
| The current serial port is occupied                                                            | Check whether another device is using the current serial port        |

### 5.3.2 Why Does Synchronization Keep Failing?

Table 5-4 Causes of Synchronization Failure

| Possible Cause              | Corrective Action                                   |
|-----------------------------|-----------------------------------------------------|
| Update mode was not entered | Hold down the UPD button and power-cycle the Device |

### 5.3.3 Why Does Erasing Keep Failing?

Possible causes are as follows:

Table 5-5 Flash Cannot Be Erased

| Possible Cause                                 | Corrective Action                  |
|------------------------------------------------|------------------------------------|
| No Flash is installed on the development board | Confirm whether Flash is installed |

| Possible Cause                | Corrective Action                  |
|-------------------------------|------------------------------------|
| Flash has a cold solder joint | Check Flash for cold solder joints |

---

## Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses, express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright ©2026 Possumic Technology Co., Ltd. All rights reserved.